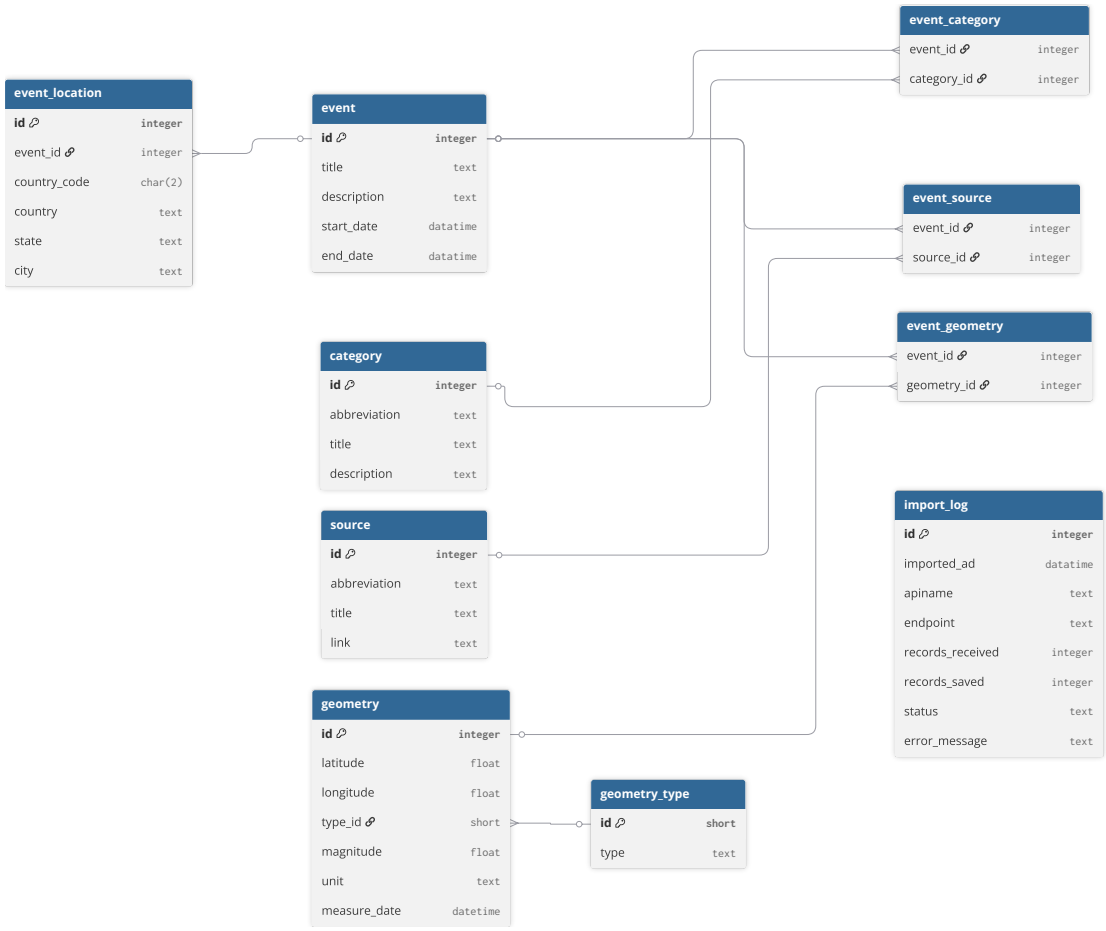
EONET API → PostgreSQL

Notebook z Projektem bazy danych oraz importem bazy danych



Encje (Tabele)

W schemacie występują następujące tabele:

- `event` - przechowuje podstawowe informacje o zdarzeniach (np. zjawiskach pobranych z API, z obciętym ID).
- `event_location` - przechowuje dane adresowe (kraj, miasto) konkretnego zdarzenia.
- `geometry` - przechowuje dokładne współrzędne geograficzne, daty pomiaru oraz dodatkowe parametry (magnitude, unit).
- `geometry_type` - tabela słownikowa przechowująca typy obiektów geometrycznych.
- `category` - tabela słownikowa przechowująca kategorie zdarzeń.
- `source` - tabela słownikowa przechowująca źródła danych dla zdarzeń.
- `event_category` - tabela asocjacyjna łącząca zdarzenia z odpowiednimi kategoriami.

- `event_source` - tabela asocjacyjna łącząca zdarzenia z odpowiednimi źródłami.
- `event_geometry` - tabela asocjacyjna łącząca zdarzenia z przypisanymi im obiektami geometrycznymi.
- `import_log` - rejestr zdarzeń przechowujący informacje o procesach importu danych z API (logi).

Lista zależności:

Zależność	Znaczenie
`event_location.event_id` → `event.id`	lokalizacja jest ściśle powiązana z konkretnym zdarzeniem
`geometry.type_id` → `geometry_type.id`	każdy rekord geometrii musi mieć określony typ
`event` → `category`	zdarzenie jest przypisane do jedenego lub wielu kategorii
`event` → `source`	informacja o zdarzeniu pochodzi z jednego lub wielu źródeł
`event` → `geometry`	zdarzenie ma przypisane jedne lub wiele dany współrzędnych

Skrypt tworzący baze danych

```
DROP VIEW IF EXISTS event_summary;
DROP TABLE IF EXISTS import_log;
DROP TABLE IF EXISTS event_category;
DROP TABLE IF EXISTS event_source;
DROP TABLE IF EXISTS event_geometry;
DROP TABLE IF EXISTS source;
DROP TABLE IF EXISTS category;
DROP TABLE IF EXISTS geometry;
DROP TABLE IF EXISTS geometry_type;
DROP TABLE IF EXISTS event_location;
DROP TABLE IF EXISTS event;

CREATE TABLE event (
        id INTEGER PRIMARY KEY, -- Przychodzi z API jako EONET_x
zapisujemy tylko x
        title  VARCHAR(255) NOT NULL, -- Do przecinka z tego co
przydchodzi w polu title
        description TEXT,
        start_date TIMESTAMP NOT NULL,
        end_date TIMESTAMP
);

CREATE TABLE event_location (
    id SERIAL PRIMARY KEY,
    event_id INTEGER NOT NULL,
    country_code CHAR(2),
    country VARCHAR(100),
```

```
    state VARCHAR(100),
    city VARCHAR(100),

    CONSTRAINT uq_event_id
                UNIQUE (event_id),

    CONSTRAINT fk_event_location_event
        FOREIGN KEY (event_id)
        REFERENCES event(id)
                ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE geometry_type (
        id SERIAL PRIMARY KEY,
        type VARCHAR(100) NOT NULL
);

CREATE TABLE geometry (
        id SERIAL PRIMARY KEY, -- Własny z sql
        latitude FLOAT NOT NULL CHECK (90 >= latitude AND latitude >=
-90), --latitude i longitude są w takich zakresach
        longitude FLOAT NOT NULL CHECK (180 >= longitude AND longitude
>= -180),
        type_id INTEGER NOT NULL,
        magnitude FLOAT,
        unit VARCHAR(50),
        measure_date TIMESTAMP NOT NULL,

        CONSTRAINT fk_geometry_type_geometry
                FOREIGN KEY (type_id)
                REFERENCES geometry_type(id)
                ON UPDATE CASCADE
                ON DELETE RESTRICT --nie chcemy usunąć typu punktu
który jest zapisany w bazie
);

CREATE TABLE category (
        id SERIAL PRIMARY KEY,
        abbreviation VARCHAR(50) NOT NULL,
        title VARCHAR(255) NOT NULL,
        description TEXT
);

CREATE TABLE source (
        id SERIAL PRIMARY KEY,
        abbreviation VARCHAR(50) NOT NULL,
        title VARCHAR(255) NOT NULL,
        description TEXT

);

CREATE TABLE event_category (
        event_id INTEGER NOT NULL,
        category_id INTEGER NOT NULL,
```

```
        CONSTRAINT pk_event_category
                PRIMARY KEY (event_id, category_id),

        CONSTRAINT fk_event_category_event
                FOREIGN KEY (event_id)
                REFERENCES event(id)
                ON UPDATE CASCADE
                ON DELETE CASCADE,

        CONSTRAINT fk_event_category_category
                FOREIGN KEY (category_id)
                REFERENCES category(id)
                ON UPDATE CASCADE
                ON DELETE CASCADE
);

CREATE TABLE event_source (
        event_id INTEGER  NOT NULL,
        source_id INTEGER NOT  NULL,


        CONSTRAINT pk_event_source
                PRIMARY KEY (event_id, source_id),

        CONSTRAINT fk_event_source_event
                FOREIGN KEY (event_id)
                REFERENCES event(id)
                ON UPDATE CASCADE
                ON DELETE CASCADE,

        CONSTRAINT fk_event_source_source
                FOREIGN KEY (source_id)
                REFERENCES source(id)
                ON UPDATE CASCADE
                ON DELETE CASCADE
);

CREATE TABLE event_geometry (
        event_id INTEGER NOT NULL,
        geometry_id INTEGER NOT NULL,


        CONSTRAINT pk_event_geometry
                PRIMARY KEY (event_id, geometry_id),

        CONSTRAINT fk_event_geometry_event
                FOREIGN KEY (event_id)
                REFERENCES event(id)
                ON UPDATE CASCADE
                ON DELETE CASCADE,

        CONSTRAINT fk_event_geometry
                FOREIGN KEY (geometry_id)
                REFERENCES geometry(id)
                ON UPDATE CASCADE
                ON DELETE CASCADE
```

```
);

CREATE TABLE import_log (
    id SERIAL PRIMARY KEY,
    imported_at TIMESTAMP NOT NULL,
    apiname VARCHAR(255) NOT NULL,
    endpoint VARCHAR(255) NOT NULL,
    records_received INTEGER NOT NULL,
    records_saved INTEGER NOT NULL,
    status VARCHAR(100) NOT NULL,
    error_message TEXT
);
```

Restrykcje nałożone na dane

- **event.id** - INTEGER pobierany API *EONET*
- **geometry.latitude** - Szerokość geograficzna od -90 do 90
- **geometry.longtitude** - Długość geograficzna od -180 do 180
- **geometry.type_id** - ON DELETE RESTRICT - Nie usuwamy typu punktu którego rodzaj istnieje w tabeli geometry

Import bibliotek

In [1]:
```
from __future__ import annotations


from sqlalchemy import URL, create_engine, text
from sqlalchemy.exc import SQLAlchemyError
import pandas as pd
```

Tworzenie schematu bazy danych

Struktura pliku konfiguracyjnego (conf.txt):

DB_USER=moj_uzytkownik

DB_PASSWORD=moje_super_tajne_haslo

DB_HOST=localhost

DB_PORT=5432

DB_NAME=moja_baza_danych

In [2]:
```
config = {}
with open("conf.txt", "r", encoding="utf-8") as f:
    for linia in f:
        linia = linia.strip()
        if not linia or linia.startswith("#"):
            continue
        klucz, wartosc = linia.split("=", 1)
        config[klucz.strip()] = wartosc.strip()
```

```
db_url = URL.create(
    drivername="postgresql+psycopg2",
    username=config.get("DB_USER"),
    password=config.get("DB_PASSWORD"),
    host=config.get("DB_HOST"),
    port=config.get("DB_PORT"),
    database=config.get("DB_NAME"),
)


from scripts import schema

engine = create_engine(db_url)
```

In [3]:
```
try:
    conn = engine.connect()
    print("[INFO] POŁACZONO DO BAZY DANYCH")
except SQLAlchemyError as e:
    print(f"[ERROR] BŁĄD POŁĄCZENIA DO BAZY DANYCH {e}")

res = conn.execute(text(schema))
conn.commit()
print("[INFO] UTWORZONO BAZĘ DANYCH")
conn.close()
```

```
[INFO] POŁACZONO DO BAZY DANYCH
[INFO] UTWORZONO BAZĘ DANYCH
```

Pobieranie danych z API

In [4]:
```
from  fetch_api import fetch_events, fetch_categories, fetch_source
DAYS = 100
LIMIT = 100
events = fetch_events(DAYS, LIMIT)
print(events[0])
sources = fetch_source()
print(sources[0])
categories = fetch_categories()
print(categories[0])
```

```
Próba 1/3 nie powiodła się: 503 Server Error: Service Unavailable for url: http
s://eonet.gsfc.nasa.gov/api/v3/events?status=all&days=100&limit=100
Próba 2/3 nie powiodła się: 503 Server Error: Service Unavailable for url: http
s://eonet.gsfc.nasa.gov/api/v3/events?status=all&days=100&limit=100
{'id': 'EONET_20292', 'title': 'Flood in Türkiye 1103920', 'description': None,
'link': 'https://eonet.gsfc.nasa.gov/api/v3/events/EONET_20292', 'closed': '2026-
06-29T00:00:00Z', 'categories': [{'id': 'floods', 'title': 'Floods'}], 'sources':
[{'id': 'GDACS', 'url': 'https://www.gdacs.org/report.aspx?eventtype=FL&eventid=1
103920'}], 'geometry': [{'magnitudeValue': None, 'magnitudeUnit': None, 'date':
'2026-06-27T20:00:00Z', 'type': 'Point', 'coordinates': [36.4534072, 37.240122
3]}]}
{'id': 'AVO', 'title': 'Alaska Volcano Observatory', 'source': 'https://www.avo.a
laska.edu/', 'link': 'https://eonet.gsfc.nasa.gov/api/v3/events?source=AVO'}
{'id': 'drought', 'title': 'Drought', 'link': 'https://eonet.gsfc.nasa.gov/api/v
3/categories/drought', 'description': 'Long lasting absence of precipitation affe
cting agriculture and livestock, and the overall availability of food and wate
r.', 'layers': 'https://eonet.gsfc.nasa.gov/api/v3/layers/drought'}
```

Zapisywanie danych do tabel

In [5]:
```
from import_danych import import_event, import_categories, import_sources, run_i

import_sources(sources, engine)
import_categories(categories, engine)
```

```
[INFO] Zaimportowano 33 źródeł
[INFO] Zaimportowano 13 kategorii
```

Import do bazdy danych

In [6]:
```
days = 200
limit= 100
run_import(engine, limit, days)
with engine.begin() as conn:
    res = pd.read_sql("SELECT * FROM import_log", con = conn)
    display(res)
```

```
[INFO] Zapisano event 20478
[INFO] Import zakończony
```

	id	imported_at	apiname	endpoint	records_received	rec
0	1	2026-06-17 12:11:01	https://eonet.gsfc.nasa.gov/api/v3/events	NASA EONERT API	100	

Skrypty do analizy lub raportów

Najczęstsze zjawiska

```
SELECT
    c.title AS category_name,
    c.abbreviation AS abbreviation,
    COUNT(ec.event_id) AS amount_of_events
FROM category c
JOIN event_category ec ON c.id = ec.category_id
GROUP BY c.title, c.abbreviation
ORDER BY amount_of_events DESC
LIMIT 10;
```

In [7]:
```
from scripts import analyze_1 as most_popular

with engine.begin() as conn:
    res = pd.read_sql(most_popular, con=conn)
    display(res)
```

	category_name	abbreviation	amount_of_events
0	Wildfires	wildfires	87
1	Floods	floods	11
2	Severe Storms	severeStorms	2

Liczba zjawisk w poszczególnych krajach

```
SELECT
    el.country AS country,
    COUNT(e.id) AS amount_of_events
FROM event e
JOIN event_location el ON e.id = el.event_id
WHERE el.country IS NOT NULL
GROUP BY el.country
ORDER BY amount_of_events DESC
LIMIT 10;
```

In [8]:
```
from scripts import analyze_2 as by_country

with engine.begin() as conn:
    res = pd.read_sql(by_country, con=conn)
    display(res)
```

	country	amount_of_events
0	Australia	25
1	United States	24
2	Congo, The Democratic Republic of the	15
3	Angola	13
4	Russian Federation	3
5	Mexico	2
6	Canada	2
7	Tanzania, United Republic of	2
8	Zambia	2
9	Turkey	2

Aktywne zjawiska

```
SELECT
    e.title AS event_title,
    e.start_date AS start_date,
    s.title AS data_source
FROM event e
JOIN event_source es ON e.id = es.event_id
JOIN source s ON es.source_id = s.id
WHERE e.end_date IS NULL
```

```
ORDER BY e.start_date DESC
LIMIT 10;
```

In [9]:
```
from scripts import analyze_3 as active

with engine.begin() as conn:
    res = pd.read_sql(active, con=conn)
    display(res)
```

	event_title	start_date	data_source
0	Green Gate (26) Wildfire	2026-06-14 19:39:00	Integrated Reporting of Wildfire Information (...
1	MIDNIGHT FIRE (53) Wildfire	2026-06-14 17:18:00	Integrated Reporting of Wildfire Information (...
2	4170 Tule Rd Wildfire	2026-06-14 16:30:00	Integrated Reporting of Wildfire Information (...
3	Rookery Wildfire	2026-06-14 10:57:00	Integrated Reporting of Wildfire Information (...
4	JUNIPER DUNES Wildfire	2026-06-14 10:25:00	Integrated Reporting of Wildfire Information (...
5	WYLY Wildfire	2026-06-13 17:54:00	Integrated Reporting of Wildfire Information (...
6	SNAKE RIVER Wildfire	2026-06-13 10:37:00	Integrated Reporting of Wildfire Information (...
7	Kilolitna Wildfire	2026-06-12 12:12:00	Integrated Reporting of Wildfire Information (...
8	Unit 16/20 Rx Prescribed Fire	2026-06-12 11:08:00	Integrated Reporting of Wildfire Information (...
9	Little Schloss RX Prescribed Fire	2026-06-10 14:51:00	Integrated Reporting of Wildfire Information (...

Zdarzenie z największą siła w danej kategorii

```
SELECT DISTINCT
    c.title AS category_name,
    e.title AS event_title,
    g.magnitude AS max_magnitude
FROM category c
JOIN event_category ec ON c.id = ec.category_id
JOIN event e ON ec.event_id = e.id
JOIN event_geometry eg ON e.id = eg.event_id
JOIN geometry g ON eg.geometry_id = g.id
WHERE e.id = (
    SELECT ec_sub.event_id
    FROM event_category ec_sub
    JOIN event_geometry eg_sub ON ec_sub.event_id = eg_sub.event_id
    JOIN geometry g_sub ON eg_sub.geometry_id = g_sub.id
    WHERE ec_sub.category_id = c.id
    ORDER BY g_sub.magnitude DESC NULLS LAST
```

```
    LIMIT 1
)
AND g.magnitude = (
    SELECT MAX(g_sub2.magnitude)
    FROM event_geometry eg_sub2
    JOIN geometry g_sub2 ON eg_sub2.geometry_id = g_sub2.id
    WHERE eg_sub2.event_id = e.id
);
```

In [10]:
```
from scripts import analyze_4 as max_from_category

with engine.begin() as conn:
    res = pd.read_sql(max_from_category, con=conn)
    display(res)
```

	category_name	event_title	max_magnitude
0	Severe Storms	Tropical Storm Cristina	40.0
1	Wildfires	Wildfire in Australia 1028883	16348.0

Najdłużej trwające zdarzenie w danej kategorii

```
WITH MaksymalneCzasy AS (
    SELECT
        c.id AS category_id,
        MAX(e.end_date - e.start_date) AS max_how_long
    FROM category AS c
    JOIN event_category AS ec ON ec.category_id = c.id
    JOIN event AS e ON e.id = ec.event_id
    WHERE e.end_date IS NOT NULL
    GROUP BY c.id
)
SELECT
    c.title AS category_name,
    e.title AS event_name,
    (e.end_date - e.start_date) AS how_long
FROM category AS c
JOIN event_category AS ec ON ec.category_id = c.id
JOIN event AS e ON e.id = ec.event_id
JOIN MaksymalneCzasy mc
    ON mc.category_id = c.id
    AND (e.end_date - e.start_date) = mc.max_how_long
```

In [11]:
```
from scripts import analyze_5 as max_time_from_category
with engine.begin() as  conn:
    res = pd.read_sql(max_time_from_category, con= conn)
    display(res)
```

	category_name	event_name	how_long
0	Floods	Flood in Mexico 1103944	2 days 04:00:00
1	Severe Storms	Tropical Storm Cristina	7 days 12:00:00
2	Wildfires	Wildfire in Russian Federation 1028929	6 days 05:00:00
3	Wildfires	Wildfire in The Democratic Republic of Congo 1...	6 days 05:00:00
4	Wildfires	Wildfire in The Democratic Republic of Congo 1...	6 days 05:00:00

Robudowany widok eventu

```sql
DROP VIEW IF EXISTS event_summary;
CREATE VIEW event_summary AS
SELECT
    e.id as event_id,
    e.title as event_title,
    e.start_date as start_date,
    e.end_date as end_date,
    el.country_code as country_code,
    el.country as country,
    el.state as state,
    el.city as city,
    g.magnitude as magnitude,
    g.unit as unit,
    g.measure_date as last_measure_date
FROM event_location AS el
JOIN event AS e ON e.id = el.event_id
JOIN event_geometry AS eg ON eg.event_id = e.id
JOIN geometry AS g ON g.id = eg.geometry_id
WHERE g.id = (SELECT eg_sub.geometry_id
    FROM event_geometry AS eg_sub
    JOIN geometry AS g_sub ON eg_sub.geometry_id = g_sub.id
    WHERE eg_sub.event_id = e.id
    ORDER BY g_sub.measure_date DESC
    LIMIT 1);
```

In [12]:
```python
from scripts import event_summary
with engine.begin() as  conn:
    conn.execute(text(event_summary))
    res = pd.read_sql("SELECT * FROM event_summary LIMIT 3", con = conn)
    display(res)
```

	event_id	event_title	start_date	end_date	country_code	country	state	
0	20292	Flood in Türkiye 1103920	2026-06-27 20:00:00	2026-06-29	TR	Turkey	Osmaniye	D
1	20520	Green Gate (26) Wildfire	2026-06-14 19:39:00	NaT	US	United States	Florida	Immol
2	20502	Wildfire in Australia 1028924	2026-06-14 19:00:00	2026-06-14	AU	Australia	Queensland	Norma

Analiza Danych i Wizualizacja

Filtry

In [13]:
```
import filtr as f
with engine.begin() as conn:
    res=f.event_category("Floods", conn)
    display(res)
```

	id	event_title	start_date	end_date	category
0	20292	Flood in Türkiye 1103920	2026-06-27 20:00:00	2026-06-29	Floods
1	20525	Flood in Mexico 1103944	2026-06-13 20:00:00	2026-06-16	Floods
2	20499	Flood in Philippines 1103942	2026-06-12 20:00:00	2026-06-14	Floods
3	20500	Flood in Türkiye 1103943	2026-06-12 20:00:00	2026-06-13	Floods
4	20465	Flood in Malaysia 1103939	2026-06-10 20:00:00	2026-06-11	Floods
5	20501	Flood in Brazil 1103940	2026-06-10 20:00:00	2026-06-12	Floods
6	20423	Flood in Ukraine 1103936	2026-06-07 20:00:00	2026-06-09	Floods
7	20449	Flood in Nicaragua 1103938	2026-06-07 20:00:00	2026-06-09	Floods
8	20393	Flood in Algeria 1103932	2026-06-06 20:00:00	2026-06-07	Floods
9	20394	Flood in China 1103933	2026-06-05 20:00:00	2026-06-07	Floods
10	20414	Flood in Canada 1103935	2026-06-05 20:00:00	2026-06-07	Floods

In [14]:
```
with engine.begin() as conn:
    res = f.event_country("China", conn)
    display(res)
```

	id	event_title	country	state	city	start_date	end_date
0	20394	Flood in China 1103933	China	Guizhou	Zhijin	2026-06-05 20:00:00	2026-06-07

In [15]:
```
with engine.begin() as conn:
    res = f.event_city("Kraków", conn)
    display(res)
```

id	event_title	country	state	city	start_date

In [16]:
```
with engine.begin() as conn:
    res = f.event_source("AVO", conn)
    display(res)
```

id	event_title	source_title	abbreviation	start_date

In [17]:
```
with engine.begin() as conn:
    res = f.event_geometry_type("Point", conn)
    display(res)
```

	id	event_title	geometry_type	latitude	longitude	measure_date
0	20292	Flood in Türkiye 1103920	Point	37.240122	36.453407	2026-06-27 20:00:00
1	20520	Green Gate (26) Wildfire	Point	26.366667	-81.166111	2026-06-14 19:39:00
2	20503	Wildfire in Australia 1028925	Point	-14.180763	142.111716	2026-06-14 19:00:00
3	20502	Wildfire in Australia 1028924	Point	-16.424747	142.412220	2026-06-14 19:00:00
4	20523	MIDNIGHT FIRE (53) Wildfire	Point	27.715278	-81.943056	2026-06-14 17:18:00
...	...	...	...	...	...	...
110	20459	Wildfire in Angola 1028873	Point	-7.385729	20.993618	2026-06-05 19:00:00
111	20460	Wildfire in The Democratic Republic of Congo 1...	Point	-6.054522	17.603371	2026-06-05 19:00:00
112	20461	Wildfire in Australia 1028885	Point	-16.349780	126.958573	2026-06-05 19:00:00
113	20477	Wildfire in The Democratic Republic of Congo 1...	Point	-6.826745	17.992982	2026-06-05 19:00:00
114	20478	Wildfire in Angola 1028892	Point	-11.114400	15.836452	2026-06-05 19:00:00

115 rows × 6 columns

In [18]:
```
with engine.begin() as conn:
    res = f.event_after_date("2024-01-01", conn)
```

```
    display(res)
```

	id	title	start_date	end_date
0	20292	Flood in Türkiye 1103920	2026-06-27 20:00:00	2026-06-29
1	20520	Green Gate (26) Wildfire	2026-06-14 19:39:00	NaT
2	20502	Wildfire in Australia 1028924	2026-06-14 19:00:00	2026-06-14
3	20503	Wildfire in Australia 1028925	2026-06-14 19:00:00	2026-06-14
4	20523	MIDNIGHT FIRE (53) Wildfire	2026-06-14 17:18:00	NaT
...	...	...	...	...
95	20459	Wildfire in Angola 1028873	2026-06-05 19:00:00	2026-06-10
96	20460	Wildfire in The Democratic Republic of Congo 1...	2026-06-05 19:00:00	2026-06-10
97	20461	Wildfire in Australia 1028885	2026-06-05 19:00:00	2026-06-11
98	20477	Wildfire in The Democratic Republic of Congo 1...	2026-06-05 19:00:00	2026-06-11
99	20478	Wildfire in Angola 1028892	2026-06-05 19:00:00	2026-06-11

100 rows × 4 columns

In [19]:
```
with engine.begin() as conn:
    res = f.event_unit("hectare", conn)
    display(res)
```

	id	event_title	magnitude	unit	measure_date
0	20502	Wildfire in Australia 1028924	10450.0	hectare	2026-06-14 19:00:00
1	20503	Wildfire in Australia 1028925	5010.0	hectare	2026-06-14 19:00:00
2	20504	Wildfire in Australia 1028926	5628.0	hectare	2026-06-13 19:00:00
3	20484	Wildfire in Zambia 1028904	7491.0	hectare	2026-06-12 19:00:00
4	20526	Wildfire in The Democratic Republic of Congo 1...	5501.0	hectare	2026-06-12 19:00:00
...	...	...	...	...	...
59	20459	Wildfire in Angola 1028873	5227.0	hectare	2026-06-05 19:00:00
60	20460	Wildfire in The Democratic Republic of Congo 1...	5914.0	hectare	2026-06-05 19:00:00
61	20461	Wildfire in Australia 1028885	5058.0	hectare	2026-06-05 19:00:00
62	20477	Wildfire in The Democratic Republic of Congo 1...	7434.0	hectare	2026-06-05 19:00:00
63	20478	Wildfire in Angola 1028892	5504.0	hectare	2026-06-05 19:00:00

64 rows × 5 columns

In [20]:
```
with engine.begin() as conn:
    res = f.category_latest_events("Wildfires", conn)
    display(res)
```

	id	event_title	category	start_date	end_date
0	20520	Green Gate (26) Wildfire	Wildfires	2026-06-14 19:39:00	NaT
1	20503	Wildfire in Australia 1028925	Wildfires	2026-06-14 19:00:00	2026-06-14
2	20502	Wildfire in Australia 1028924	Wildfires	2026-06-14 19:00:00	2026-06-14
3	20523	MIDNIGHT FIRE (53) Wildfire	Wildfires	2026-06-14 17:18:00	NaT
4	20516	4170 Tule Rd Wildfire	Wildfires	2026-06-14 16:30:00	NaT
5	20514	PV Wildfire	Wildfires	2026-06-14 14:52:00	2026-06-15
6	20513	Rookery Wildfire	Wildfires	2026-06-14 10:57:00	NaT
7	20517	JUNIPER DUNES Wildfire	Wildfires	2026-06-14 10:25:00	NaT
8	20504	Wildfire in Australia 1028926	Wildfires	2026-06-13 19:00:00	2026-06-14
9	20519	WYLY Wildfire	Wildfires	2026-06-13 17:54:00	NaT
10	20515	26 Wildfire	Wildfires	2026-06-13 15:16:00	2026-06-13
11	20518	SNAKE RIVER Wildfire	Wildfires	2026-06-13 10:37:00	NaT
12	20526	Wildfire in The Democratic Republic of Congo 1...	Wildfires	2026-06-12 19:00:00	2026-06-14
13	20484	Wildfire in Zambia 1028904	Wildfires	2026-06-12 19:00:00	2026-06-13
14	20522	Kilolitna Wildfire	Wildfires	2026-06-12 12:12:00	NaT
15	20524	Unit 16/20 Rx Prescribed Fire	Wildfires	2026-06-12 11:08:00	NaT
16	20468	Wildfire in Australia 1028898	Wildfires	2026-06-11 19:00:00	2026-06-12
17	20467	Wildfire in Brazil 1028897	Wildfires	2026-06-11 19:00:00	2026-06-12
18	20466	Wildfire in The Democratic Republic of Congo 1...	Wildfires	2026-06-11 19:00:00	2026-06-11
19	20450	Wildfire in Australia 1028879	Wildfires	2026-06-11 19:00:00	2026-06-11

In [21]:
```
with engine.begin() as conn:
    res_active = f.event_status("active", conn)
    display(res_active)
```

	id	title	start_date	end_date
0	20520	Green Gate (26) Wildfire	2026-06-14 19:39:00	None
1	20523	MIDNIGHT FIRE (53) Wildfire	2026-06-14 17:18:00	None
2	20516	4170 Tule Rd Wildfire	2026-06-14 16:30:00	None
3	20513	Rookery Wildfire	2026-06-14 10:57:00	None
4	20517	JUNIPER DUNES Wildfire	2026-06-14 10:25:00	None
5	20519	WYLY Wildfire	2026-06-13 17:54:00	None
6	20518	SNAKE RIVER Wildfire	2026-06-13 10:37:00	None
7	20522	Kilolitna Wildfire	2026-06-12 12:12:00	None
8	20524	Unit 16/20 Rx Prescribed Fire	2026-06-12 11:08:00	None
9	20446	Little Schloss RX Prescribed Fire	2026-06-10 14:51:00	None
10	20442	South Fork Wildfire	2026-06-09 20:08:00	None
11	20521	Bear Wildfire	2026-06-09 17:45:00	None
12	20447	APQ Area 11 Scrub RX 0608 Prescribed Fire	2026-06-09 11:20:00	None
13	20444	South Mountain Wildfire	2026-06-09 00:15:00	None
14	20443	PUTAH Wildfire	2026-06-08 14:35:00	None
15	20387	Tower Wildfire	2026-06-07 15:24:00	None
16	20448	LXR Lower Third RX 0609 Prescribed Fire	2026-06-07 14:01:00	None
17	20388	Raven Creek Wildfire	2026-06-06 20:44:00	None
18	20389	CHESTNUT Wildfire	2026-06-06 14:14:00	None

In [22]:
```
with engine.begin() as conn:
    res = f.event_country_category("Australia", "Wildfires", conn)
    display(res)
```

	id	event_title	country	city	category	start_date
0	20502	Wildfire in Australia 1028924	Australia	Normanton	Wildfires	2026-06-14 19:00:00
1	20503	Wildfire in Australia 1028925	Australia	Aurukun	Wildfires	2026-06-14 19:00:00
2	20504	Wildfire in Australia 1028926	Australia	Jabiru	Wildfires	2026-06-13 19:00:00
3	20450	Wildfire in Australia 1028879	Australia	Mission River	Wildfires	2026-06-11 19:00:00
4	20468	Wildfire in Australia 1028898	Australia	Humpty Doo	Wildfires	2026-06-11 19:00:00
5	20485	Wildfire in Australia 1028905	Australia	Ngukurr	Wildfires	2026-06-11 19:00:00
6	20486	Wildfire in Australia 1028916	Australia	Humpty Doo	Wildfires	2026-06-11 19:00:00
7	20487	Wildfire in Australia 1028917	Australia	Derby	Wildfires	2026-06-11 19:00:00
8	20469	Wildfire in Australia 1028899	Australia	Milingimbi	Wildfires	2026-06-10 19:00:00
9	20470	Wildfire in Australia 1028900	Australia	Humpty Doo	Wildfires	2026-06-10 19:00:00
10	20489	Wildfire in Australia 1028918	Australia	Maningrida	Wildfires	2026-06-10 19:00:00
11	20529	Wildfire in Australia 1028942	Australia	Derby	Wildfires	2026-06-10 19:00:00
12	20426	Wildfire in Australia 1028855	Australia	Humpty Doo	Wildfires	2026-06-09 19:00:00
13	20451	Wildfire in Australia 1028880	Australia	Humpty Doo	Wildfires	2026-06-09 19:00:00
14	20452	Wildfire in Australia 1028881	Australia	Kununurra	Wildfires	2026-06-09 19:00:00
15	20506	Wildfire in Australia 1028927	Australia	Humpty Doo	Wildfires	2026-06-09 19:00:00
16	20507	Wildfire in Australia 1028928	Australia	Cossack	Wildfires	2026-06-09 19:00:00
17	20396	Wildfire in Australia 1028846	Australia	Maningrida	Wildfires	2026-06-08 19:00:00
18	20453	Wildfire in Australia 1028882	Australia	Wadeye	Wildfires	2026-06-08 19:00:00
19	20454	Wildfire in Australia 1028883	Australia	Aurukun	Wildfires	2026-06-08 19:00:00
20	20474	Wildfire in Australia 1028901	Australia	Maningrida	Wildfires	2026-06-08 19:00:00

	id	event_title	country	city	category	start_date
21	20427	Wildfire in Australia 1028868	Australia	Maningrida	Wildfires	2026-06-07 19:00:00
22	20455	Wildfire in Australia 1028884	Australia	Aurukun	Wildfires	2026-06-07 19:00:00
23	20361	Wildfire in Australia 1028814	Australia	Wadeye	Wildfires	2026-06-05 19:00:00
24	20461	Wildfire in Australia 1028885	Australia	Kununurra	Wildfires	2026-06-05 19:00:00

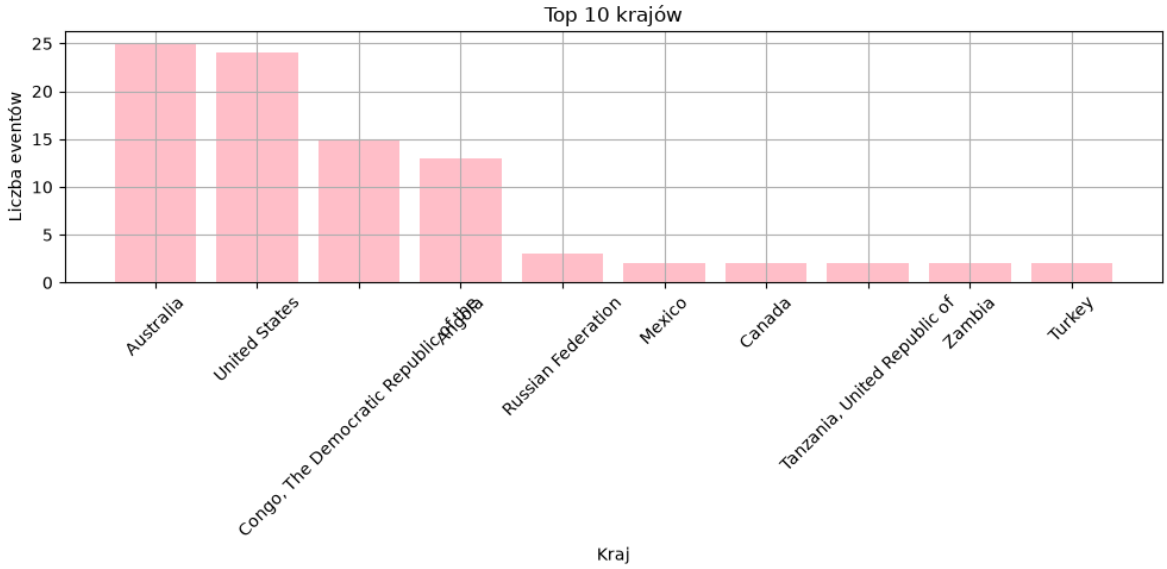
Wizualizacja

In [23]:
```
import pandas as pd
import matplotlib.pyplot as plt

with engine.begin() as conn:

    res = pd.read_sql("""
        SELECT
            country,
            COUNT(*) AS amount_of_events
        FROM event_location
        WHERE country IS NOT NULL
        GROUP BY country
        ORDER BY amount_of_events DESC
        LIMIT 10
    """, conn)

plt.figure(figsize=(10,5))
plt.bar(res["country"], res["amount_of_events"],color="pink")
plt.title("Top 10 krajów")
plt.xlabel("Kraj")
plt.ylabel("Liczba eventów")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid()
plt.show()
```



In [24]:
```
with engine.begin() as conn:

    res = pd.read_sql("""
        SELECT
            c.title AS category,
            COUNT(*) AS amount_of_events
        FROM category c
        JOIN event_category ec
            ON c.id = ec.category_id
        GROUP BY c.title
        ORDER BY amount_of_events DESC
    """, conn)

plt.figure(figsize=(10,5))
plt.bar(res["category"], res["amount_of_events"],color="green")
plt.title("Najpopularniejsze kategorie zjawisk")
plt.xlabel("Kategoria")
plt.ylabel("Liczba eventów")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [25]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

```
with engine.begin() as conn:

    res = pd.read_sql("""
        SELECT
            latitude,
            longitude
        FROM geometry
        WHERE latitude BETWEEN -90 AND 90
          AND longitude BETWEEN -180 AND 180
    """, conn)

img = plt.imread("image.png")

fig, ax = plt.subplots(figsize=(16, 8))

ax.imshow(
    img,
    extent=[-180, 180, -90, 90],
    aspect="auto"
)

ax.scatter(
    res["longitude"],
    res["latitude"],
    color="red",
    s=12,
    alpha=0.7
)

ax.set_xlim(-180, 180)
ax.set_ylim(-90, 90)

ax.axis("off")

plt.subplots_adjust(
    left=0,
    right=1,
    top=1,
    bottom=0
)

plt.show()
```

